PROJECT FLOW

The Next-Generation Project Management & Tracking Application

Document Version: 1.0.0	**Status:** Production Ready / Active
Author / Lead: Nirav Tandel	**Date:** November 2026
Repository: project-flow	**Stack:** React 18 + FastAPI + PostgreSQL/SQLite

1. Project Details & Objectives

Project Flow is an enterprise project tracking and task orchestration platform designed for software teams and engineering managers. It provides a structured environment to organize projects, break down deliverables into subtask checklists, manage file assets, monitor team progress metrics in real time, and auto-complete projects when all goals are met.

Field	Details / Value
Project Name	Project Flow - The Project Management Application
Core Vision	High-productivity task & project management with real-time subtask checklists and analytics.
Target Audience	Engineering teams, product owners, agile squads, and individual builders.
Current Status	In Progress (All core modules functional with 100% test passing)
Priority	High
Project Timeline	October 25, 2026 – November 20, 2026
Security Model	Argon2id password hashing + JWT sessions + strict user_id row-level authorization.

2. Project Requirements & Feature Specifications

2.1 Functional Requirements:

- **User Authentication:** Secure registration, login, JWT token issuance, session persistence, and strict multi-tenant project separation.
- **Project Management:** CRUD operations with fields (name, description, start_date, due_date), statuses (Not Started, In Progress, On Hold, Completed), priorities (Low, Medium, High), search, and filtering.
- **Task System:** Nested tasks inside projects with title, description, priority, due date, status (To Do, In Progress, Done), and one-click completion.
- **Subtask Checklists:** Dynamic subtasks inside each task, toggleable in one click, without clunky modal round-trips.
- **Document Repository:** Project-level document storage supporting PDF, docs, images, and spreadsheets with download and delete capabilities.
- **Progress Tracking:** Task progress: *(subtasks done ÷ total subtasks)*; Project progress: *(tasks done ÷ total tasks)*; Automatic status switch to **Completed** when all tasks finish.
- **Executive Dashboard:** Live project metrics, status breakdowns, deadlines due today & this week, and overdue task count.

2.2 Non-Functional Requirements:

- **Performance:** Sub-100ms API response latency, debounced search, and optimized indexed database queries.
- **Data Integrity:** Foreign keys with ON DELETE CASCADE to maintain database hygiene across tasks, subtasks, and files.
- **Responsive UX:** Fluid Tailwind CSS layouts optimized for smartphones, tablets, and widescreen monitors.

3. System Architecture & Tech Stack

Project Flow follows a modular 3-tier architecture ensuring clean separation between presentation, domain logic, and data storage:

Tier	Technologies	Key Responsibilities & Components
Presentation Layer	React 18, TypeScript, Tailwind CSS, Vite, Lucide Icons, React Router 6	Client-side routing, responsive UI modals, optimistic checklist updates, progress bars, dropzone file upload, and dashboard cards.
Application Layer	FastAPI, Python 3.12, Pydantic v2, Python-Multipart, PyJWT	REST routing, JWT auth guards, auto-completion business logic, file stream endpoints, schema validation, exception handlers.
Persistence Layer	SQLAlchemy 2.0 ORM, PostgreSQL / SQLite, Alembic migrations	User-scoped data queries, cascade relationships, index optimizations on foreign keys (project_id, task_id, user_id).
Storage Layer	Local File System / Scalable Object Store (UUID Hashed)	Project-isolated file directories (`uploads/project_{id}/`), preventing name collisions and security path-traversal exploits.

4. End-to-End Process Flows

4.1 Authentication & Multi-Tenancy Isolation:

1. User submits credentials to POST /api/v1/auth/login.
2. Backend verifies password hash using Argon2id and returns a signed JWT token.
3. Frontend stores token in localStorage and attaches it to all HTTP requests via Authorization: Bearer.
4. All project queries strictly append where user_id = current_user.id, guaranteeing tenant isolation.

4.2 Project & Task Lifecycle with Auto-Completion:

1. User creates a Project with target dates and priority.
2. Tasks are created with checklists of subtasks.
3. Checking subtasks updates individual task progress bars in real time.
4. When all tasks in a project are marked Done, the backend trigger _check_project_completion() automatically promotes the project status to **Completed**.

4.3 Document Management Workflow:

1. Client uploads document via multipart/form-data to /api/v1/documents/projects/{id}/upload.
2. Backend validates project access and generates a unique UUID stored name.
3. File stream is written to uploads/project_{project_id}/ and metadata is recorded in the database.
4. Files can be streamed with original download filenames or deleted with automatic file cleanup.

5. Comprehensive REST API Endpoints Reference

Base URI prefix: /api/v1. All endpoints except registration and login require JWT authentication.

Module	HTTP Method & Path	Description & Parameters
Auth	`POST /auth/register`	Create a new user account with email, name, password.
Auth	`POST /auth/login`	OAuth2 password request; returns JWT access token.
Auth	`GET /auth/me`	Returns current authenticated user profile.
Projects	`GET /projects/`	List user projects with optional status and search.
Projects	`POST /projects/`	Create project (name, description, priority, dates).
Projects	`GET /projects/{id}`	Retrieve project details by ID with calculated stats.
Projects	`PUT /projects/{id}`	Update project metadata and target dates.
Projects	`DELETE /projects/{id}`	Delete project, cascading to tasks, subtasks, and files.
Tasks	`GET /tasks/?project_id={id}`	List tasks belonging to specified project.
Tasks	`POST /tasks/`	Create task with optional initial subtasks array.
Tasks	`GET /tasks/{id}`	Retrieve single task with full subtasks list.
Tasks	`PUT /tasks/{id}`	Update task title, description, priority, and due date.
Tasks	`PATCH /tasks/{id}/status`	Update task status (triggers project auto-completion check).
Tasks	`DELETE /tasks/{id}`	Delete task and cascade-remove its subtasks.
Subtasks	`POST /subtasks/`	Add checklist item under a task (task_id, title).
Subtasks	`PATCH /subtasks/{id}/toggle`	Toggle subtask is_done boolean state.
Subtasks	`DELETE /subtasks/{id}`	Delete subtask item from checklist.
Documents	`POST /documents/projects/{id}/upload`	Upload file attachment (multipart/form-data).
Documents	`GET /documents/projects/{id}`	List all file metadata records attached to project.
Documents	`GET /documents/{id}/download`	Stream download file with original filename header.
Documents	`DELETE /documents/{id}`	Delete document metadata & remove file from disk storage.
Dashboard	`GET /dashboard/summary`	Executive metrics: projects by status, due today/week, overdue.

6. Database Entity Relationships & Constraints

Table Name	Key Columns & Types	Relationships & Foreign Key Rules
users	id (PK), email (UQ, indexed), hashed_password, full_name, created_at	1-to-Many with projects, project_documents.
projects	id (PK), user_id (FK), name, description, status (Enum), priority (Enum), start_date, due_date, created_at	Belongs to users. 1-to-Many with tasks and project_documents (CASCADE delete).
tasks	id (PK), project_id (FK), title, description, status (Enum), priority (Enum), due_date, completed_at	Belongs to projects. 1-to-Many with subtasks (CASCADE delete).
subtasks	id (PK), task_id (FK), title, is_done (Boolean), created_at	Belongs to tasks. Index on task_id for rapid checklist rendering.
project_documents	id (PK), project_id (FK), user_id (FK), filename, stored_name, file_size, content_type, created_at	Belongs to projects & users. Maps database records to disk files.